ThingIndex: Inteligência Documental com Proveniência Verificável sobre Modelo Relacional

Charles Eduardo Mello Guimarães[1]

[1] Instituto Federal do Paraná (IFPR) – Campus Palmas
Curso de Sistemas de Informação
Palmas – PR – Brasil

charles.eduardo.mello.guimaraes@hotmail.com

Resumo. *Quem consulta leis, normas e manuais precisa conferir a resposta no documento original. Este trabalho, conduzido por* Design Science Research*, apresenta o ThingIndex, sistema que guarda o conteúdo de PDFs em tabelas relacionais. O objetivo é verificar se é possível obter fontes conferíveis usando só o modelo relacional. Cada parte é ligada ao documento por chave estrangeira e guarda a página de onde veio, gravada na indexação. A busca é feita por palavras (BM25), e cada resposta mostra suas fontes, lidas do banco. O sistema foi avaliado de forma exploratória, para caracterizar onde acerta e onde falha. Em três acervos públicos, com perguntas escritas depois do desenvolvimento, a taxa de acerto foi de 68,1% com a pergunta original. Com a expansão da pergunta, recurso experimental em que um LLM acrescenta termos técnicos antes da busca, chegou a 88,7%; o recurso fica desligado por padrão, e o ganho foi medido só em documentos públicos. Sem expansão, o acerto foi de 88,2% quando a pergunta usava os termos do documento e de 38,7% quando não usava. No acervo de desenvolvimento, a busca enviou ao modelo cerca de 9 vezes menos* tokens *que o documento inteiro, na mediana, com o mesmo acerto (71 de 72 perguntas nos dois casos). A resposta à pergunta de pesquisa é sim para a origem das fontes, em parte para a busca e não para a conferência: o sistema citou, com a página certa, um artigo de lei já revogada, e a fonte citada não mostra a revogação.*

Palavras-chave: recuperação de informação, proveniência de dados, auditabilidade, geração aumentada por recuperação, modelagem relacional.

Abstract. *People who consult laws, regulations and manuals need to check the answer against the original document. Following Design Science Research, this work presents ThingIndex, a system that stores the content of PDFs in relational tables. The goal is to verify whether checkable sources can be obtained using only the relational model. Each part is linked to its document by a foreign key and keeps the page it came from, recorded at indexing. Retrieval is keyword-based (BM25), and each answer shows its sources, read from the database. The system was evaluated in an exploratory way, to characterize where it succeeds and where it fails. On three public collections, with questions written after development, accuracy was 68.1% with the original question. With query expansion, an experimental feature in which an LLM adds technical terms before*

retrieval, it reached 88.7%; the feature is off by default, and the gain was measured only on public documents. Without expansion, accuracy was 88.2% when the question used the document's terms and 38.7% when it did not. On the development collection, retrieval sent the model about 9 times fewer tokens than the whole document, in the median, with the same accuracy (71 of 72 questions in both cases). The answer to the research question is yes for source provenance, partly for retrieval and no for verification: the system cited, with the correct page, an article of a repealed law, and the cited source does not show the repeal.

Keywords: information retrieval, data provenance, auditability, retrieval-augmented generation, relational modeling.

1. Introdução

Muitas perguntas de trabalho têm resposta em um documento que já existe. Um técnico de segurança do trabalho precisa confirmar quem paga o equipamento de proteção; uma secretaria acadêmica precisa localizar a carga horária de um curso no projeto pedagógico; um servidor do setor de compras de uma prefeitura precisa saber se a modalidade "tomada de preços" ainda pode ser usada, caso que este trabalho mostra ser o mais difícil (Seção 8.6). A resposta está em leis, normas e regulamentos, guardados em pastas ou em sistemas que indexam o nome do arquivo, mas não o conteúdo. Achar a cláusula exige abrir o arquivo certo e lê-lo, e uma resposta errada gera retrabalho ou decisão mal fundamentada.

Os modelos de linguagem de grande escala (LLMs), que estão por trás dos assistentes de conversa, mudaram a expectativa: em vez de uma lista de arquivos, espera-se uma resposta pronta. A técnica mais usada para isso é a geração aumentada por recuperação (RAG): antes de responder, o sistema busca trechos relevantes nos documentos e os entrega ao modelo junto com a pergunta [Lewis et al. 2020]. Quem responde por uma decisão, porém, precisa conferir a resposta no original. Sem a página de origem, o leitor tem de refazer a busca para conferir; e uma resposta errada que cita uma fonte plausível pode passar sem ser notada.

Neste trabalho, um sistema é *auditável* quando cada fonte que ele mostra é *verificável*: aponta o documento e a página de onde veio, lidos do banco de dados, sem depender do texto escrito pelo modelo. Na literatura de bancos de dados, isso se chama proveniência *where* [Buneman et al. 2001]. Chama-se de *inteligência documental* a capacidade de um sistema organizar e recuperar o conteúdo dos documentos, além do nome e dos metadados. A pergunta de pesquisa é: *é possível sustentar um sistema de inteligência documental com auditabilidade utilizando exclusivamente modelagem relacional, sem depender de bancos vetoriais ou grafos como camada primária de recuperação?*

A expressão "sustentar um sistema com auditabilidade" foi desdobrada em três condições que os dados podem confirmar ou não, retomadas na Discussão: (a) toda fonte mostrada ao usuário tem origem lida do banco; (b) a busca por palavras sobre as partes do documento guardadas em tabelas, com a nota calculada na aplicação e sem vetores, encontra o trecho certo em perguntas que não foram usadas no desenvolvimento; e (c) conferir a fonte permite confirmar que a resposta está certa. A condição (c) vai além

da definição de fonte verificável e foi incluída porque é o que importa para quem usa a resposta.

Para responder, foi construído o ThingIndex. O sistema extrai a estrutura dos PDFs (páginas, parágrafos com posição, tabelas e sumário), guarda essa estrutura em tabelas do PostgreSQL, busca os trechos por palavras, com o algoritmo BM25, e devolve com cada fonte a indicação que permite abri-la no documento. Na pergunta, o LLM só entra no final, para redigir a resposta a partir das fontes encontradas; na indexação, ele pode gerar resumos que também entram na busca. A Figura 1 resume o caminho de uma pergunta. A contribuição principal é o sistema e o modo como ele guarda a origem de cada fonte, pela chave estrangeira até a página. A avaliação das Seções 7 e 8 é exploratória e foi feita em acervos pequenos. A comparação com a busca densa, que procura por significado com vetores, se limita a um *corpus* de controle: cinco textos curtos e fictícios, montados para o teste. Não usar um banco de vetores, a escolha mais comum em RAG, foi uma decisão de projeto, para manter um único banco a operar; a Seção 8.1 mede parte do que se perde com isso.

Pergunta do usuário $\rightarrow$ **busca BM25** nas tabelas da coleção $\rightarrow$ até **9 fontes**, cada uma com documento e página lidos do banco $\rightarrow$ o **LLM** redige a resposta só a partir delas $\rightarrow$ a tela mostra a resposta e **abre cada fonte no PDF**, com o parágrafo destacado

Figura 1. Caminho de uma pergunta no ThingIndex. As fontes são escolhidas pela busca e lidas do banco antes de o modelo escrever. Na configuração padrão, o modelo só influencia essa escolha pelos resumos que gerou na indexação (Seção 6.4).

Em áreas reguladas e na administração pública, quem decide precisa mostrar em que norma a decisão se apoia, e uma resposta sem fonte não serve para isso. Os trabalhos mais próximos deixam a origem das fontes a cargo de quem monta o sistema ou não avaliam como ela é guardada (Seção 4). Um sistema que depende só de um banco relacional e de um modelo aberto também pode rodar dentro da própria organização, sem enviar os documentos a terceiros, o que facilita atender à LGPD; esse uso, porém, não foi testado aqui.

As contribuições são:

1. o sistema, implementado e testado, em que cada unidade recuperável é ligada ao documento por chave estrangeira e carrega a página e, nos parágrafos, a região, com a chave conferida por teste (Seção 6.3);
2. um protocolo para avaliar o sistema: hipóteses registradas antes de medir, gabaritos travados por *hash*, a mesma pergunta escrita em duas redações (uma vez com as palavras do documento e outra com as palavras de quem não o leu) e registro das correções feitas depois de medir;
3. resultados exploratórios em português, em sete acervos (seis de PDFs reais e um sintético, de controle), sobre acerto, origem das fontes e custo, incluindo um erro que a fonte verificável não detecta: a resposta que cita, com a página certa, uma norma revogada.

O texto segue esta ordem: objetivos (Seção 2), referencial teórico (Seção 3), trabalhos relacionados (Seção 4), metodologia (Seção 5), o sistema (Seção 6), o desenho

da avaliação (Seção 7), os resultados (Seção 8), a discussão (Seção 9) e a conclusão (Seção 10). Diagramas, telas e tabelas completas estão nos anexos, no repositório do projeto (Apêndice A).

2. Objetivos

O objetivo geral é projetar, implementar e avaliar o ThingIndex, um sistema de inteligência documental que organiza e recupera o conteúdo de coleções de PDFs com fontes verificáveis, e caracterizar sua recuperação diante da busca densa e da entrega do documento inteiro a um LLM.

Os objetivos específicos são:

1. modelar em tabelas relacionais as estruturas extraídas de PDFs (páginas, parágrafos, tabelas, sumário e trechos), com a coordenada de origem de cada unidade exigida pelo esquema;
2. implementar o sistema completo, com indexação assíncrona resiliente a falhas, recuperação por coleção e *chat* que exibe as fontes;
3. definir as medidas de avaliação (acerto, página nas fontes, origem no índice e custo) e um protocolo com gabaritos independentes da saída do sistema;
4. caracterizar a recuperação lexical diante da densa, exata e aproximada, em qualidade de ordenação num *corpus* de controle e em estabilidade;
5. caracterizar, em acerto, proveniência e custo, o contexto recuperado diante da entrega do documento inteiro ao modelo;
6. medir o sistema em acervos de domínios distintos, comparando o acerto da resposta com a presença da página certa entre as fontes;
7. testar intervenções dirigidas às causas de erro encontradas.

Os objetivos 1 e 2 respondem à condição (a) da Introdução. O 5 também serve a ela: com o documento inteiro, a página citada só existe no texto do modelo, e com a busca ela vem do banco; o 5 mede ainda o custo em *tokens*. Os objetivos 6 e 7 respondem à condição (b), e o 4 mede, no *corpus* de controle, o que se perde ao dispensar os vetores. A condição (c) é respondida pelos dados do objetivo 6, em especial pelo acervo de legislação. O objetivo 3 fornece o instrumento de medida.

3. Referencial Teórico

Os termos mais usados no artigo estão resumidos na Tabela 1, para consulta durante a leitura.

3.1. Busca por palavras e busca por significado

A recuperação de informação estuda como encontrar, numa coleção, os itens que atendem a uma necessidade de informação [Manning et al. 2008]. A forma mais antiga é a busca por palavras, ou *lexical*. O BM25, usado no ThingIndex, dá uma nota a cada trecho a partir de três ideias [Robertson e Zaragoza 2009]: cada palavra da pergunta encontrada no trecho soma pontos; uma palavra rara na coleção vale mais que uma comum, porque diz mais sobre o assunto (num acervo de normas de segurança, “EPI” (equipamento de proteção individual) vale mais que “trabalho”); e repetir a palavra ajuda cada vez menos, enquanto trechos longos são penalizados, para que um trecho não vença só por ter mais palavras. Os parâmetros k_1 e b regulam essas duas últimas ideias. A primeira versão

Tabela 1. Termos usados no artigo, no sentido em que são usados aqui.

Termo	Significado
LLM	Modelo de linguagem de grande escala: lê um texto de entrada (o *prompt*) e escreve a resposta.
Token	Pedaço de palavra que o LLM processa. O custo de uma chamada cresce com os *tokens* enviados, e cada modelo aceita um máximo deles (a *janela*).
RAG	Geração aumentada por recuperação: buscar trechos nos documentos e entregá-los ao LLM junto com a pergunta.
Busca lexical	Busca por palavras: um trecho é bom se contém as palavras da pergunta. O BM25 é o algoritmo usado.
Busca densa	Busca por significado: pergunta e trecho são convertidos em listas de números (vetores), e trechos de sentido parecido ficam próximos.
Índice aproximado	Estrutura que acelera a busca densa examinando só parte dos vetores, ao custo de às vezes perder o mais próximo.
Unidade, fonte	Unidade é cada parte do PDF guardada numa linha de tabela (página, parágrafo, linha de tabela, seção, trecho). Fonte é a unidade mostrada ao usuário junto da resposta: uma seção, uma página ou uma linha de tabela.
C1 e bateria	C1 é o *corpus* de controle, com cinco textos fictícios. A bateria reúne os três acervos com perguntas escritas depois do desenvolvimento (segurança do trabalho, produto e consumidor, licitações), medidos juntos.
Braço	Cada configuração comparada num experimento (por exemplo, BM25 contra busca densa).
Gabarito	Lista de perguntas, cada uma com a página onde está a resposta e o padrão que uma resposta certa precisa conter.
Literal, paráfrase e par	As duas redações de uma mesma pergunta: com as palavras do documento (literal) e com as palavras de quem não leu o documento (paráfrase). As duas juntas formam um *par*.
Por resposta e por pergunta	Na bateria e no acervo de ensino, cada pergunta foi feita três vezes (nos de legislação e do edital, uma vez; Tabela 4). "Por resposta" conta cada uma das três; "por pergunta" só conta acerto quando as três acertam.
Página nas fontes e página no texto	Página nas fontes: a página certa está entre as fontes que a tela mostra; vem do banco e não pode ser inventada. Página no texto: a página certa está escrita no texto da resposta; vem do modelo e pode estar errada.

do ThingIndex só contava ocorrências, sem dar peso à raridade. No BEIR, conjunto de testes em domínios variados, o BM25 superou a maior parte dos métodos densos da época [Thakur et al. 2021].

A fraqueza da busca por palavras é a diferença de vocabulário entre quem pergunta e quem escreveu o documento. Furnas et al. [1987] mostraram que duas pessoas raramente escolhem a mesma palavra para a mesma coisa: quem pergunta "quem paga o capacete e a luva?" não usa as palavras da norma, que fala em equipamento de proteção individual. Uma solução é a *expansão da pergunta*, que acrescenta à busca termos relacionados. Com LLMs, pode-se pedir ao próprio modelo esses termos, que é o método de Jagerman et al. [2023] e o usado neste trabalho.

A outra família é a busca *densa*, ou por significado. Um modelo treinado converte cada texto numa lista de centenas de números, chamada vetor, de modo que textos de sentido parecido fiquem com vetores próximos. A pergunta também é convertida em vetor, e a busca devolve os trechos mais próximos dela. A vantagem é encontrar "carro" quando se pergunta "automóvel". Este trabalho usa o `multilingual-e5-base` [Wang et al. 2024], um modelo multilíngue que gera vetores de 768 números.

Comparar a pergunta com todos os vetores guardados é exato, mas fica lento com milhões deles. Por isso se usam *índices aproximados*, como o IVF e o HNSW: o IVF separa os vetores em grupos, como gavetas, e na consulta abre só as gavetas mais próximas; o HNSW liga cada vetor a seus vizinhos e caminha por essa rede até a região da pergunta. Os dois podem deixar de fora um vizinho que a busca exata acharia, e a documentação do pgvector, extensão que acrescenta esses índices ao PostgreSQL, avisa que o resultado pode diferir do da busca exata [Kane 2026]. Por fim, listas de resultados de métodos diferentes podem ser combinadas pela fusão por posição recíproca (RRF), que dá a cada item, em cada lista, a nota $1/(k + \text{posição})$, com k constante (60 neste trabalho), e soma essas notas [Cormack et al. 2009].

3.2. Geração aumentada por recuperação

Um LLM responde a partir do que viu no treinamento e, sem uma fonte externa, produz com frequência afirmações fluentes e falsas, o que se chama de alucinação [Ji et al. 2023]. A RAG, proposta por Lewis et al. [2020], combina um buscador com o gerador: os trechos encontrados entram no *prompt* junto com a pergunta, e o modelo é instruído a responder a partir deles. Na forma mais comum, os documentos são cortados em blocos de tamanho fixo, convertidos em vetores e guardados num banco de vetores [Gao et al. 2023]; nesse corte, a ligação de cada bloco com a página do original fica a cargo de quem monta o sistema.

Alguns limites conhecidos dessa técnica ajudam a ler os experimentos deste trabalho. O modelo aproveita pior a informação que está no meio de um texto longo [Liu et al. 2024]. Quando as fontes entregues se contradizem, como uma lei e a lei que a revogou, o modelo não resolve o conflito de forma confiável [Xu et al. 2024]. E o modelo tende a concordar com a premissa da pergunta mesmo quando ela é falsa, comportamento chamado de bajulação [Sharma et al. 2024]. Com modelos que aceitam textos muito longos, uma alternativa à RAG é entregar o documento inteiro; Li et al. [2024] concluíram que o documento inteiro acerta mais quando há recursos para ele, mas a recu-

peração custa muito menos.

3.3. Modelo relacional e proveniência

No modelo relacional de Codd [1970], os dados ficam em tabelas, e as ligações entre elas são garantidas por chaves: uma chave estrangeira faz o banco recusar uma linha que aponte para outra que não existe. Quando a restrição existe de fato no esquema, quem garante isso é o próprio banco, mesmo que o programa tenha algum erro (a Seção 6.3 mostra um caso em que ela faltava).

Proveniência é a informação sobre a origem de um dado. Buneman et al. [2001] distinguem a proveniência *where*, que diz onde está o valor de origem, da *why*, que diz quais dados justificam um resultado. No ThingIndex, o interesse é a *where*: cada unidade guardada aponta documento e página e, nos parágrafos, a região que ocupa na página. É preciso separar isso de outra pergunta: a frase da resposta é sustentada pela fonte citada? Essa é a noção de *atribuição* [Rashkin et al. 2023]. Saber de onde a fonte veio é condição para conferir a atribuição, mas não a substitui: o sistema pode apontar a página certa de um trecho que não sustenta o que a resposta diz.

3.4. Extração da estrutura de PDFs

Um PDF guarda o texto como caracteres posicionados na página, sem marcar o que é parágrafo, título ou tabela. Para recuperar essa estrutura, há dois caminhos: regras sobre a posição e a forma do texto, ou modelos treinados sobre a diagramação, como o LayoutLM [Xu et al. 2020], que combina o texto com sua posição na página e precisa ser ajustado com exemplos anotados para cada tarefa. Páginas escaneadas são imagens e precisam antes de reconhecimento óptico de caracteres (OCR). O ThingIndex usa regras (Seção 6.4), que dispensam dados anotados e dão sempre o mesmo resultado para o mesmo PDF, ao custo de errar em diagramações incomuns.

3.5. Integração assíncrona

Indexar um PDF leva tempo, então o envio só registra o pedido, e um processo separado faz o trabalho depois, avisado por uma fila de mensagens. O problema é que gravar no banco e publicar na fila são duas escritas sem transação comum: se o sistema cair entre elas, o documento fica registrado sem que ninguém seja avisado. O padrão *transactional outbox* resolve isso gravando a mensagem numa tabela, na mesma transação do registro, e publicando-a depois [Richardson 2018].

4. Trabalhos Relacionados

A Tabela 2 compara o ThingIndex com os trabalhos mais próximos quanto a duas escolhas: como as fontes são escolhidas e de onde vem a indicação de origem que o usuário vê.

Bibliotecas como o LangChain servem para montar aplicações com LLMs e deixam a origem das fontes a cargo de quem as usa; o ThingIndex é uma aplicação completa que fixa uma escolha (unidades da estrutura do PDF, com página) e a avalia. O GraphRAG trata de perguntas que exigem visão de conjunto do acervo, mas depende do LLM já na indexação, e o que recupera pode ser um resumo gerado. O PageIndex é o mais próximo: também usa a estrutura do PDF e dispensa vetores, mas é o modelo que escolhe a seção.

Tabela 2. Comparação com trabalhos relacionados.

Trabalho	Como escolhe as fontes	Origem da fonte
LangChain [Chase 2022] e bibliotecas semelhantes	O desenvolvedor escolhe o corte em blocos e o buscador	Os metadados que o desenvolvedor anexar ao bloco
GraphRAG [Edge et al. 2024]	Grafo de entidades e resumos construídos por LLM	Resumos gerados pelo LLM, no lugar do texto original
PageIndex [VectifyAI 2025]	O LLM navega a árvore de seções do PDF	A seção escolhida pelo modelo; sem avaliação publicada
Assistentes comerciais	Fechado	Referência ao documento, sem garantia de que se possa inspecionar
ThingIndex	BM25 sobre unidades guardadas em tabelas. A escolha das fontes é do BM25; o LLM pode gerar resumos que entram no índice e, com a expansão ligada, sugerir palavras de busca	Documento ligado por chave estrangeira; página gravada na indexação e lida do banco

Assistentes comerciais que aceitam PDFs atendem muitos usuários ao mesmo tempo, o que o ThingIndex não faz; ele difere deles por poder rodar inteiramente na instalação da organização, com um modelo aberto, e por ter a garantia de origem visível no esquema e no código. O uso na própria instalação não foi avaliado: os experimentos usaram um provedor externo. Não houve comparação experimental com esses produtos nem com o PageIndex.

Quanto à avaliação, ferramentas como o RAGAS [Es et al. 2024] usam o próprio LLM como juiz das respostas. Este trabalho preferiu gabaritos escritos à mão, com página-alvo e padrão de resposta, para que o juiz não seja outro modelo. Nenhum dos trabalhos da Tabela 2 mede como a origem das fontes é guardada num sistema completo; aqui isso é medido em PDFs reais em português.

5. Metodologia

O trabalho é uma pesquisa aplicada conduzida segundo a *Design Science Research* (DSR), em que se constrói um artefato para resolver um problema relevante e se avalia o artefato com rigor [Hevner et al. 2004]. Seguiu-se o processo de seis atividades de Peffers et al. [2007]:

1. **Identificação do problema**: consulta a acervos normativos sem fonte verificável (Seção 1). Não houve levantamento com usuários; o problema foi caracterizado por casos de uso típicos e pela literatura.
2. **Objetivos da solução**: mostrar fontes com documento e página lidos do banco, achar o trecho certo em perguntas não usadas no desenvolvimento e enviar ao modelo menos *tokens* que o documento inteiro (Seção 2).

3. **Projeto e desenvolvimento**: o ThingIndex (Seção 6).
4. **Demonstração**: uso sobre seis acervos de PDFs reais e um *corpus* sintético de controle (Seção 7).
5. **Avaliação**: exploratória, com experimentos de busca, de origem das fontes e de geração da resposta, limitados ao necessário para testar as três condições da Introdução (Seção 8). A utilidade para usuários finais não foi avaliada.
6. **Comunicação**: este artigo e o repositório do projeto, com código, dados, *scripts* de análise e anexos.

O desenvolvimento foi feito em ciclos: cada experimento que revelava uma limitação gerava uma mudança no sistema, medida no ciclo seguinte. Esses ciclos usaram um único acervo, o de ensino, que por isso funcionou como *conjunto de desenvolvimento*: o sistema foi ajustado com base nas suas 72 perguntas, e seus resultados tendem a ser otimistas. Os gabaritos dos demais acervos foram escritos depois, sobre documentos ainda não usados, e funcionam como *conjunto de teste*. Para evitar que os ajustes sucessivos inflassem os números, foram adotadas as práticas a seguir:

- **Pré-registro**: nas duas intervenções testadas (Seções 8.8 e 8.9), a hipótese, o critério de sucesso e a análise foram escritos e versionados antes do código; a primeira desviou do plano e é tratada como exploratória (Seção 8.8).
- **Gabaritos travados**: cada gabarito registra o *hash* do seu conteúdo, calculado pela função de *hash* SHA-256, e o programa de avaliação recusa um gabarito alterado sem atualizar o *hash*, o que deixa toda alteração visível no histórico.
- **Números gerados por *script***: os resultados ficam em arquivos CSV, uma linha por pergunta e repetição, e as tabelas do artigo são geradas ou conferidas a partir deles.
- **Registro das correções feitas depois de medir**: toda alteração de gabarito ou de critério feita depois de uma medição foi registrada com data, motivo e efeito nos números (Anexo A8). As que mudam números deste artigo são citadas onde o número aparece.

6. O ThingIndex

6.1. Requisitos

Os requisitos estão na Tabela 3. Cada requisito funcional corresponde a rotas da API, a interface de programação pela qual o *frontend* acessa o sistema, e essas rotas são cobertas por testes automatizados. O texto completo dos requisitos está no Anexo A1, e o diagrama de casos de uso, no Anexo A2. Recuperação determinística (RNF08) quer dizer que a mesma pergunta sobre o mesmo índice traz as mesmas fontes na mesma ordem.

Quanto à privacidade, com um modelo rodando na própria instalação nenhum conteúdo sai dela; com um provedor externo, como o usado nos experimentos, a pergunta e os trechos recuperados são enviados a ele, e cabe à organização a base legal para isso, nos termos da Lei Geral de Proteção de Dados (LGPD). Nesta versão o sistema é de uso individual, sem compartilhamento de coleções.

6.2. Arquitetura

O sistema tem quatro partes: o *frontend*, em Next.js e React; a API, em Go; o *indexer*, também em Go, que processa os documentos; e a infraestrutura, com PostgreSQL 16, a

Tabela 3. Requisitos funcionais (RF) e não funcionais (RNF), resumidos do Anexo A1.

Id	Requisito funcional	Id	Requisito não funcional
RF01	Cadastro, *login*, redefinição de senha e perfil	RNF01	Senhas guardadas com bcrypt
RF02	Categorias e coleções	RNF02	Sessão por *token* assinado (JWT)
RF03	Envio de PDF para uma coleção	RNF03	Credenciais de provedor cifradas (AES-GCM)
RF04	Indexação em segundo plano (páginas, parágrafos, tabelas, sumário)	RNF04	Limite de requisições em autenticação, envio e *chat*
RF05	Resumos por LLM (opcional)	RNF05	Integridade referencial e transações (PostgreSQL)
RF06	Conteúdo indexado de uma coleção por API	RNF06	Camadas: *handlers*, casos de uso e repositórios
RF07	Respostas a perguntas, com as fontes	RNF07	Execução reproduzível com Docker Compose
RF08	Abertura da fonte no PDF, na página e no parágrafo	RNF08	Recuperação determinística
RF09	Provedores de LLM e de armazenamento, com credencial cifrada	RNF09	Uso de modelo local, sem enviar conteúdo a terceiros
RF10	Andamento da indexação, *tokens* e custo estimado		
RF11	Histórico de ações (auditoria)		
RF12	Reprocessamento de falhas		

fila RabbitMQ e um armazenamento de arquivos compatível com o Amazon S3 (MinIO no ambiente local). O Go foi escolhido por gerar um binário único, simples de implantar, e por rodar tarefas concorrentes no mesmo processo; no *indexer*, o consumo da fila, o despachante do *outbox* e a verificação de documentos presos rodam lado a lado. O PostgreSQL foi escolhido porque a integridade referencial e as transações são a base da proposta. O RabbitMQ confirma o recebimento de cada mensagem e mantém filas de nova tentativa e de mensagens descartadas, que o envio descrito a seguir usa. A API é organizada em camadas: os *handlers* HTTP validam a entrada, os casos de uso contêm a regra de negócio e os repositórios isolam o acesso ao banco. Os provedores de LLM e de armazenamento ficam atrás de interfaces e podem ser trocados sem mudar os casos de uso; os experimentos usaram o Ollama em nuvem, por um provedor externo (Seção 7).

No envio de um documento, a API grava o arquivo e, numa única transação, registra o documento, o pedido de indexação na tabela de *outbox* e o evento de auditoria. Um processo despachante lê os pedidos pendentes e os publica no RabbitMQ, marcando-os como publicados só depois que a fila confirma o recebimento. Se algo falhar no meio, o pedido é publicado de novo, e o *indexer* ignora o pedido repetido de um documento já indexado. Um documento que continua na fila 30 minutos depois de o pedido ser publicado ganha um novo pedido; um que fica mais de 30 minutos em processamento, como quando o *indexer* para no meio, é marcado como falho, assim como um que falha em

todas as tentativas, com a causa registrada. O diagrama de sequência desse fluxo está no Anexo A3.

6.3. Modelo de dados e proveniência

O esquema tem 25 tabelas (Anexo A9). Quatro formam o núcleo (usuários, categorias, coleções e documentos). Seis guardam a estrutura extraída, uma por tipo de unidade: páginas, parágrafos, tabelas, nós do sumário, trechos e metadados. As demais servem à operação: *outbox*, tentativas de processamento, auditoria, provedores, conversas e *cache*. A Figura 2 mostra o recorte com o caminho da proveniência (Anexo A4).

Páginas, parágrafos, tabelas e trechos carregam sua origem: o documento, o número da página, a posição na ordem de leitura e, nos parágrafos, a região (a caixa delimitadora) que o parágrafo ocupa na página, extraída do PDF junto com o texto. As seções do sumário guardam a página inicial e a final, e os metadados guardam só o documento. O vínculo com o documento é uma chave estrangeira: o banco recusa um parágrafo que aponte para um documento inexistente. A página e a região, por outro lado, são gravadas pelo *indexer* na mesma linha, a página numa coluna e a região num campo JSON, e o banco não confere se a página existe no documento; fora da tabela de páginas, a coluna da página também aceita valor vazio. Um trecho recebe a página do seu primeiro parágrafo, de modo que um trecho que atravessa duas páginas aponta só a primeira. A página mostrada ao usuário é a gravada na indexação e lida do banco, e o modelo não tem como alterá-la.

Ao revisar o código para este artigo, descobriu-se que essa chave estrangeira não existia: a biblioteca de mapeamento objeto-relacional usada, o GORM, só cria a restrição quando o modelo declara a associação, e os seis modelos de estrutura não a declaravam. A coluna existia e era obrigatória, mas o banco não verificava o vínculo. A correção declarou a associação, removeu linhas órfãs na migração e acrescentou dois testes de integração: um confere no catálogo do PostgreSQL que as seis restrições existem, e o outro, que o banco recusa um parágrafo sem documento; só a partir daí o banco passou a conferir o vínculo. A correção não muda o que o *indexer* grava nem como a busca escolhe as fontes. Por isso não altera os resultados da Seção 8, embora parte deles tenha sido medida antes da correção.

6.4. Indexação

O *indexer* extrai o texto com a ferramenta `pdftotext`, que entrega cada página separada e a posição de cada bloco e linha. A partir disso, regras sobre a posição e a forma do texto (caixa alta, numeração) montam os parágrafos na ordem de leitura, detectam tabelas pelo alinhamento das colunas, reconstroem a hierarquia de seções pelos títulos e pela numeração, e agrupam os parágrafos em trechos por seção. Uma tabela que atravessa páginas é dividida em uma parte por página, para que cada célula tenha página. Páginas escaneadas passam por reconhecimento óptico de caracteres (OCR). Nelas, a região só existe quando o OCR devolve a posição do texto; quando não devolve, o parágrafo é gravado sem região, e o sistema não inventa uma.

Quando há um provedor de LLM configurado, o *indexer* também gera um resumo de cada página e de cada seção. Esse *enriquecimento* vem ligado por padrão, pode ser desligado nas preferências e fica em *cache*, identificado pelo conteúdo da página, para

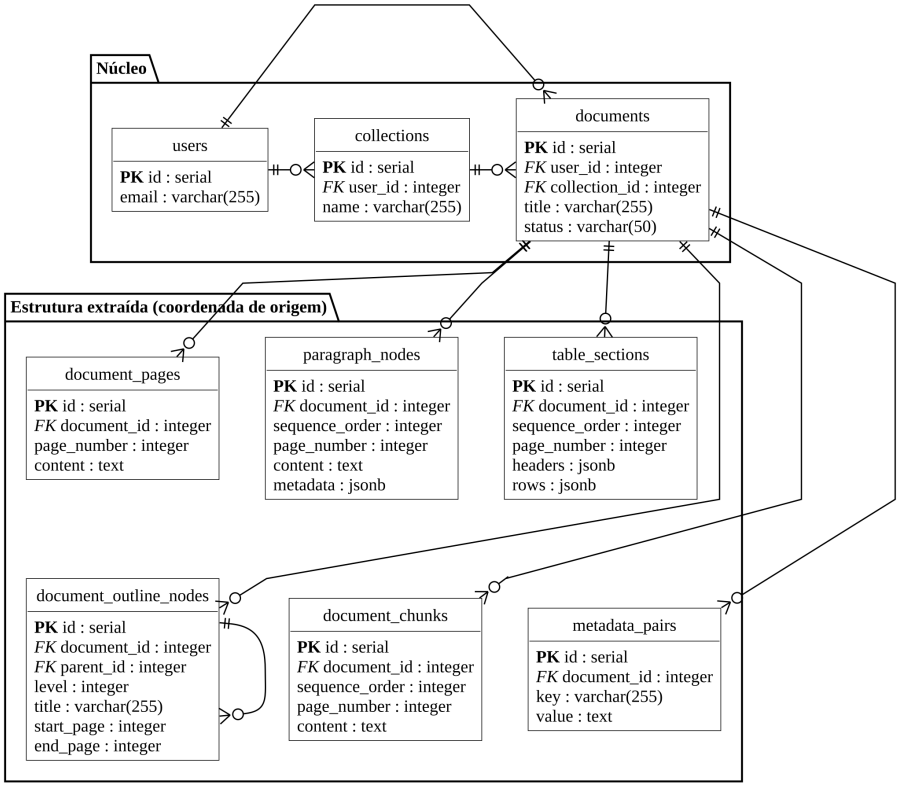


Figura 2. Recorte do esquema com o caminho da proveniência. Cada tabela da estrutura extraída aponta para o documento por chave estrangeira (`document_id`); páginas, parágrafos, tabelas e trechos guardam também a página (`page_number`), as seções guardam a página inicial e a final, e a região do parágrafo fica no campo `metadata`. O esquema completo, com as 25 tabelas, está no Anexo A9.

não ser pago de novo ao reindexar. Na configuração padrão, o resumo da página entra na busca junto com o texto da página, como descrito a seguir; a Seção 8.8 mede o efeito disso.

6.5. Recuperação e chat

Quando o usuário pergunta algo sobre uma coleção, a recuperação acontece em três passos (Anexo A5). Primeiro, o sistema carrega as unidades da coleção sempre com um número fixo de consultas ao banco (nove), qualquer que seja o número de documentos; um teste de integração conta as consultas para 1 e para 20 documentos e falha se o número mudar. Cada consulta tem um limite, como 5.000 páginas e 20.000 parágrafos por coleção. As linhas são lidas na ordem dos documentos, do primeiro enviado ao último, e, dentro de cada um, na ordem das páginas ou da leitura. Acima do limite, as unidades dos documentos enviados por último ficam fora da busca sem aviso ao usuário; o evento só é registrado em *log*, o que é uma limitação desta versão.

Segundo, as unidades recebem nota por BM25 ($k_1 = 1{,}2$; $b = 0{,}75$), com acentos e maiúsculas ignorados; para as palavras mais longas da pergunta, a busca inclui também a forma no singular ou no plural, obtida por regras simples. Na configuração padrão, usada nos experimentos de geração, exceto nos braços comparados na Seção 8.8, a nota

da página considera o texto da página e o resumo gerado na indexação. Páginas, seções, trechos e linhas de tabela recebem nota em separado, e as notas são combinadas com pesos fixos por tipo de unidade, escolhidos durante o desenvolvimento (Apêndice B).

Terceiro, o sistema seleciona até três linhas de tabela e até seis fontes entre seções e páginas, com no máximo três seções, isto é, até nove fontes por resposta. A nota dos trechos não gera fonte: ela só ajuda a escolher o documento. Empates na nota são desfeitos pelo documento e, dentro dele, pelo número da página ou pelo identificador da seção, e as linhas de tabela empatadas mantêm a ordem de leitura; assim, a ordem é sempre a mesma para o mesmo índice. Por fim, as fontes são reordenadas pelo número de termos da pergunta que cada uma contém, mantendo a ordem anterior nos empates; uma linha de tabela só fica na frente quando contém pelo menos metade desses termos.

O banco guarda as unidades e a origem de cada uma; a nota BM25 é calculada pela API, a cada pergunta, sobre as unidades carregadas, sem um índice de busca guardado à parte. Isso evita que um índice fique desatualizado em relação às tabelas. O tempo de cálculo cresce com o tamanho da coleção e não foi medido em separado; nas coleções usadas, de até 10 documentos, ele não impediu os experimentos. Para acervos maiores, o índice de texto do próprio PostgreSQL fica como trabalho futuro.

As fontes selecionadas são enviadas à tela antes da resposta e, em seguida, entram no *prompt* do modelo, com a instrução de responder apenas a partir delas e de dizer quando a informação não consta. De cada fonte, vai ao modelo a linha de tabela ou o texto da página, cortado em 4.000 caracteres, até um total de 24.000 caracteres; as fontes que passam desse total entram só com um trecho de 220 caracteres. Além das fontes, o *prompt* leva uma lista de até seis documentos da coleção, com o resumo de cada um gerado na indexação, e as últimas mensagens da conversa, quando a tela as envia (nos experimentos, cada pergunta abriu uma conversa nova). Esse conteúdo não é fonte e não aparece como tal na tela. A expansão da pergunta (experimental; Seção 8.9) também existe no código.

6.6. Interface

A tela principal é o *workspace* de uma coleção (Figura 3). Os documentos aparecem como páginas de PDF, e o *chat* fica na lateral. As fontes aparecem numeradas abaixo da resposta, com documento e página. Ao selecionar uma delas, a tela vai até a página e destaca sobre o PDF o parágrafo dessa página que contém mais palavras da pergunta, usando a região gravada na indexação; sem região gravada, a página abre sem destaque. A ligação entre cada frase da resposta e a fonte que a sustenta não é exibida: o modelo cita a página no texto, e cabe ao leitor associá-la à fonte. As demais telas estão no Anexo A6.

6.7. Verificação do software

O *backend* tem 159 arquivos de teste, com 732 funções de teste; 26 desses arquivos são de testes de integração, quase todos contra PostgreSQL, RabbitMQ ou MinIO reais; a cobertura medida foi de 56,0% das instruções. O *frontend* tem 85 arquivos de teste unitário e 16 arquivos, com 33 casos, de teste de interface em navegador. Há ainda um teste de caos, que derruba, um de cada vez e com o sistema em funcionamento, o RabbitMQ durante um envio, o PostgreSQL e o armazenamento de arquivos, e confere que o sistema volta a funcionar sem ser reiniciado; num cenário à parte, vinte envios simultâneos de PDF para

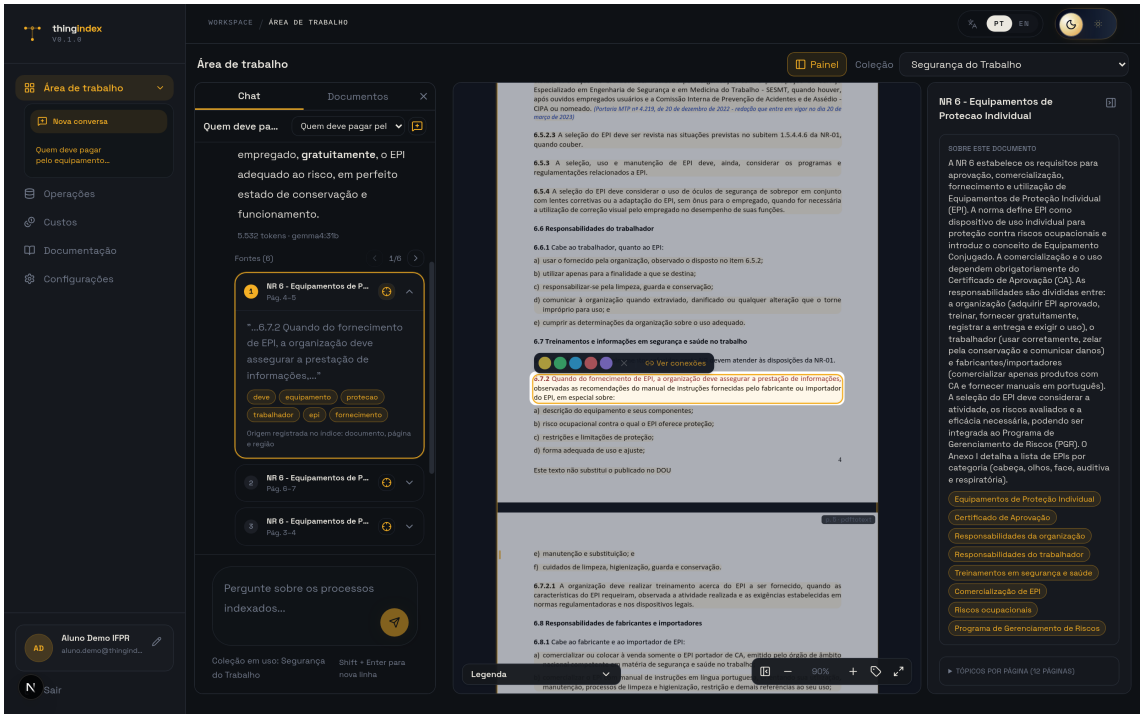


Figura 3. *Workspace* de uma coleção de normas de segurança do trabalho: *chat* com a primeira fonte expandida (à esquerda), PDF na página citada com o parágrafo destacado (ao centro) e o resumo do documento gerado na indexação (à direita).

a mesma coleção terminam indexados. As versões finais de todos os testes, inclusive o de caos (16 de 16 verificações), foram executadas localmente.

A integração contínua do repositório roda, a cada *pull request* e a cada mudança na versão principal, os testes do *backend*, 22 dos arquivos de teste de integração, os do *frontend* e o teste de caos. Ficam fora da integração contínua os testes de integração que chamam um LLM real ou exigem o emulador do Google Cloud Storage, e o teste do *benchmark* de indexação.

7. Desenho da Avaliação

A avaliação é exploratória: descreve o comportamento do sistema em acervos pequenos, com gabaritos escritos e corrigidos por uma só pessoa, e seus números indicam tendências, sem pretensão de generalização. Ela responde a três perguntas, nesta ordem, com resultados na Seção 8: a busca por palavras é competitiva com a busca densa (Seções 8.1 a 8.3)? As fontes entregues têm origem verificável (Seção 8.4)? E, no sistema completo, com o modelo redigindo a resposta, quais são o acerto, a citação e o custo (Seções 8.5 a 8.10)? A primeira pergunta serve à condição (b) da Introdução; a segunda, à (a); e a terceira, à (b) e à (c). A Tabela 5, no início daquela seção, resume as respostas.

A Tabela 4 resume os acervos usados; C1 é o nome dado ao *corpus* de controle, o único acervo montado só para o teste. A *bateria* reúne três acervos com perguntas escritas depois do desenvolvimento, medidos juntos e com três repetições: segurança do trabalho; produto e consumidor; e licitações. O Anexo A10 traz, para cada acervo, os PDFs indexados, exceto o do edital municipal, que não está versionado, e um arquivo

com todas as perguntas, a resposta esperada e a página onde ela está.

Tabela 4. Acervos e gabaritos usados na avaliação. Repetições: quantas vezes cada pergunta foi feita.

Acervo	Documentos	Perguntas	Repetições	Seção
C1, recuperação	5 textos fictícios elaborados para o teste, de 111 a 166 palavras (jurídico, licitação, contratual, normativo e operacional)	18	–	8.1
Ensino	10 PDFs: 8 do IFPR e 2 normas externas (Lei 11.788/2008 e Resolução CNE/CES 5/2016)	72	3	8.5
Legislação	Leis 8.666/1993 e 14.133/2021	19	1	8.6
Segurança do trabalho	5 PDFs de 4 Normas Regulamentadoras (NR-1 em duas redações)	23	3	8.7
Produto e consumidor	Código de Defesa do Consumidor, manual de monitor, termo de garantia	23	3	8.7
Licitações	Leis 8.666, 14.133 e 10.520	25	3	8.7
Edital municipal	1 edital de concurso, 52 páginas	18	1	8.10

Gabaritos. Todas as perguntas foram escritas a partir dos PDFs, sem consultar o sistema. Cada uma declara o documento e a página onde está a resposta e um padrão de texto (expressão regular) que a resposta certa precisa conter; quando necessário, declara também padrões que invalidam a resposta, como os de uma recusa (“não há informação sobre isso”). Um teste automatizado confere que uma recusa genérica não conta como acerto em nenhuma pergunta que tenha resposta. Na bateria e no edital, a maior parte das perguntas foi escrita duas vezes: uma versão *literal*, com as palavras do documento, e uma *paráfrase*, com as palavras de quem não leu o documento, como “Quem deve fornecer o EPI ao trabalhador e em que condições?” e “Quem paga o capacete e a luva que eu uso no serviço?”. As duas versões da mesma pergunta formam um *par*. Na bateria, as 9 perguntas negativas não têm par. Os pares existem porque quem escreve a pergunta logo depois de ler a página tende a copiar as palavras dela, e isso favorece a busca por palavras; comparar as duas versões mostra o tamanho desse favorecimento.

As três medidas principais. O *acerto* é a porcentagem das respostas corretas. Uma resposta é correta quando satisfaz o gabarito; nas perguntas *negativas*, que não têm resposta no acervo, quando declara que a informação não consta. Como cada pergunta da bateria foi feita três vezes, as porcentagens contam respostas; nos testes estatísticos, a pergunta conta como certa só quando acertou nas três vezes. A *página nas fontes* é a porcentagem das respostas em que a página do gabarito estava entre as fontes mostradas na tela, lidas do banco; conta-se também a página seguinte quando a frase da fonte continua nela, como a tela exibe. É o que o leitor consegue conferir. O *custo* é o número de

tokens enviados ao modelo por pergunta. Só na comparação com a entrega do documento inteiro, em que não há fontes, usa-se também a *página no texto*, escrita pelo modelo na resposta. Nenhuma dessas medidas confere se o trecho sustenta a frase da resposta.

Métrica de busca. Em C1, usa-se a medida mais simples: em quantas das 16 consultas com resposta o primeiro resultado da lista era o texto certo (P@1, na literatura); as demais medidas estão no Anexo A7.

Origem no índice. Mede-se a porcentagem de unidades indexadas com *origem completa* no banco, isto é, com documento, página, posição e conteúdo preenchidos. Confere-se também se o texto de cada parágrafo extraído aparece, letra por letra (ignorando maiúsculas e espaços), no documento de onde o sistema diz que ele veio.

Custo e modelo. Não se usou valor em reais no custo porque o preço muda com o provedor e com o tempo. Todos os experimentos de geração usaram o mesmo modelo aberto, `gemma4:31b`, servido pelo Ollama em nuvem, isto é, por um provedor externo; o uso com um modelo rodando na própria instalação é possível, mas não foi medido. A *temperatura* (o grau de variação que o modelo se permite ao escrever) foi a padrão do sistema, 0,2 nas respostas dadas pelo *chat*; a versão exata do modelo no provedor não foi fixada. As três repetições medem a variação que resta.

Estatística. Os resultados são dados em contagem ("13 de 16") ou, na bateria, em porcentagem das respostas. Nas comparações entre duas condições sobre as mesmas perguntas da bateria, usou-se o teste de McNemar [McNemar 1947]; em C1, o teste de aleatorização pareado, com correção para múltiplas comparações (Anexo A7). Uma diferença é tratada como estatisticamente significativa quando o valor-p fica abaixo de 0,05; os testes e os intervalos de confiança estão no Anexo A7.

8. Resultados

Os resultados a seguir verificam o sistema descrito na Seção 6. As medições completas estão nos anexos e nas tabelas de apoio do repositório (Apêndice A). A Tabela 5 resume a resposta a cada pergunta da avaliação.

8.1. Busca por palavras contra busca densa

C1 é um *corpus* de controle: cinco textos curtos e fictícios, com 18 consultas, das quais 16 têm resposta e duas não. As consultas são literais (palavras tiradas do documento), paráfrases (a mesma necessidade com outras palavras) ou ruidosas (com erros de digitação e sem acentos). Os seis métodos foram comparados nas mesmas consultas, na escolha entre os 5 textos inteiros. Na escolha entre as 19 seções, só foram medidas as três buscas por palavras (a contagem, o BM25 e a combinação dos dois). A Tabela 6 traz os principais.

Entre os 5 textos, o BM25 acertou o primeiro resultado em 13 das 16 consultas e a busca densa em 11; a versão aproximada da busca densa acertou tantas vezes quanto a exata (11 de 16). Com tão poucas consultas, a diferença não é estatisticamente significativa (Anexo A7). Por tipo de consulta, o BM25 acertou as 5 literais, 5 das 7 paráfrases e 3 das 4 ruidosas; a densa, 3 das 5 literais, 4 das 7 paráfrases e as 4 ruidosas, e os dois erraram juntos só 1 consulta (literais e paráfrases em porcentagem no Anexo A7, Tabela 2; as ruidosas, por consulta, em `docs/article/results/results.csv`, no repositório). Entre as 19 seções, mais próximo do que o sistema faz, o BM25 acertou 12 e

Tabela 5. Resumo dos resultados. As porcentagens são da bateria e contam respostas (cada pergunta foi feita três vezes); "x de y" conta consultas, unidades, perguntas ou pares.

O que se perguntou	Resultado	Seção
Busca por palavras ou por significado: qual recupera mais vezes o texto certo?	Sem diferença detectável: 13 de 16 contra 11 de 16 consultas no primeiro resultado	8.1
Cada unidade tem a origem gravada no banco?	Sim: o documento é exigido pelo esquema, e a página, gravada na indexação; em C1 (sem páginas), 155 de 155, 85 delas metadados	8.4
Buscar trechos custa menos que mandar o PDF inteiro?	Sim, no acervo de desenvolvimento: cerca de 9 vezes menos *tokens* enviados para responder, na mediana das perguntas com resposta, com 71 de 72 perguntas certas contra 71 de 72	8.5
Quanto o sistema acerta em perguntas novas?	68,1% das respostas, cerca de 7 em 10	8.7
A redação da pergunta muda o acerto?	Sim: 88,2% com as palavras do documento e 38,7% com outras; 15 pares discordantes, todos a favor da literal, com o gabarito revisado	8.7
Expandir a pergunta corrige isso?	Em parte: de 68,1% para 88,7% no total, e as paráfrases chegam a 83,9%; 15 perguntas ganhas e nenhuma perdida	8.9
Fonte certa garante resposta certa?	Não: no acervo de legislação, respostas erradas citaram a página certa da lei revogada	8.6

Tabela 6. Busca em C1: em quantas das 16 consultas com resposta o primeiro resultado era o texto certo. Entre as 19 seções, só as buscas por palavras foram medidas. Os seis métodos e as demais medidas estão no Anexo A7.

	Primeiro resultado certo	
Método	Entre 5 textos	Entre 19 seções
Contagem de palavras (versão anterior)	12 de 16	9 de 16
BM25 (usado no sistema)	13 de 16	12 de 16
Busca densa	11 de 16	–
BM25 + busca densa (híbrida)	13 de 16	–

a contagem de palavras, 9; essa diferença também não é estatisticamente significativa, e trocar a contagem pelo BM25 nos experimentos seguintes foi uma decisão de projeto.

8.2. Quando não há resposta

Nas duas consultas de C1 sem resposta, todos os métodos devolveram alguma fonte; quem reconhece a falta de resposta é a instrução dada ao modelo. Com a busca, o modelo declarou que a informação não consta nas 5 negativas do acervo de ensino (Seção 8.5) e em todas as 27 respostas negativas da bateria; com o documento inteiro, errou uma das 5 negativas do ensino.

8.3. Estabilidade dos resultados

Cada método de C1 foi executado 30 vezes: a contagem, o BM25, a combinação dos dois e a busca densa exata deram as mesmas listas em todas as execuções, mas o índice aproximado, na configuração do teste (muitos grupos e um só consultado), mudou as listas ao ser refeito ou com outra ordem de inserção. No pgvector, com 1.000, 5.000 e 20.000 vetores sintéticos, o IVFFlat na configuração recomendada pela documentação foi estável nos três tamanhos, e o HNSW com os parâmetros padrão variou a partir de 5.000 vetores (Anexo A7, Tabela 5). A estabilidade depende, portanto, do índice e da configuração escolhidos.

8.4. Origem das fontes no índice

Na versão atual, as 155 unidades indexadas de C1 têm a origem completa. A primeira medição deu 51 de 155. A correção do sistema, que deu coluna de página aos trechos, levou a 70; a maior parte da diferença veio de ajustar a métrica, pois as 85 unidades de metadados, que descrevem o documento inteiro e não têm página, passaram a contar como origem completa (Anexo A7, em porcentagem; em contagem, nas tabelas de apoio do Apêndice A). Como C1 não tem paginação, toda unidade fica na página 1: a medida mostra que a origem está gravada, não que a página esteja certa, o que é conferido por testes unitários e, indiretamente, pela página entre as fontes nas seções seguintes. Os 20 parágrafos extraídos de C1 aparecem, letra por letra, no documento de origem.

8.5. Trechos recuperados contra documento inteiro

No braço **T** (*trechos*), o avaliador usa a mesma rota de *chat* da tela. No braço **F** (*full*), o mesmo modelo recebe, sem busca, o texto completo do documento que contém a resposta, com as páginas marcadas (nas negativas, cinco documentos); como já recebe o documento certo, F mostra o melhor caso da entrega do documento inteiro.

Os dois braços não usaram exatamente a mesma configuração: T passou pelo *chat* (temperatura 0,2 e o *prompt* do sistema), e F chamou o modelo diretamente, com outro *prompt*; o código pede temperatura 0,0 para F e 0,2 para T.

O acervo de ensino tem 10 PDFs e 72 perguntas e é o de desenvolvimento, por isso o resultado é otimista: T foi ajustado nessas perguntas e o gabarito teve quatro versões no período (e uma revisão posterior, registrada no Anexo A8), enquanto F foi executado numa única rodada, com três repetições, sem ajuste no sistema. Os números de T são da última execução com três repetições, depois do ajuste na ordem das fontes; um ajuste

posterior, que leva junto até 600 caracteres da página seguinte quando a frase é interrompida na mudança de página, foi medido numa repetição, sem perder acerto, e está ativo em todas as medições da Seção 8.6 em diante.

Acerto e custo. T e F acertaram 71 das 72 perguntas nas três repetições: T errou a mesma pergunta de prosa nas três, e F, a mesma negativa. Pergunta a pergunta, nas 67 perguntas com resposta, F enviou ao modelo, na mediana, 9,1 vezes mais *tokens* que T, numa razão que variou de 0,7 a 41,9 vezes conforme o tamanho do documento. Em 22 das 72 perguntas, sobre documentos curtos, a busca enviou mais *tokens* que o documento inteiro. A conta é só dos *tokens* enviados para responder e não inclui os resumos gerados na indexação, pagos uma vez por documento. Em T, cada resposta levou, na mediana, cerca de 1,2 s, da pergunta ao fim do texto, incluindo a busca e o modelo no provedor.

Origem. Nas 201 respostas com página-alvo (as negativas não têm uma), a página no texto esteve correta em 189 de 201 respostas em T e em 189 de 201 em F, aceitando também o número de página impresso no próprio documento, que nem sempre coincide com a posição da página no PDF (sem essa tolerância, os números estão no apoio dos resultados, Apêndice A). Os dois braços acertaram a página no texto o mesmo número de vezes; a diferença está em de onde vem a página. Em F, ela só existe no texto do modelo, e, na pergunta que F errou, a resposta atribui a uma página um dado inexistente. Em T, a página nas fontes vem do banco, e a página certa esteve entre as fontes em 195 de 201 respostas; o modelo, porém, também escreve páginas no texto, e essas podem estar erradas.

8.6. Legislação com norma revogada

O acervo de legislação tem a Lei 8.666/1993 e a Lei 14.133/2021, que a revogou; a resposta certa depende da data e do contrato, e os dois textos usam quase o mesmo vocabulário. Das 19 perguntas, as *divergentes* exigem distinguir as duas leis, e responder só pela revogada conta como erro. O sistema acertou 15 das 19 perguntas; nas 5 divergentes, acertou 3. Perguntado se a modalidade "tomada de preços" existe, o sistema respondeu que sim e citou corretamente o art. 22 da Lei 8.666, sem dizer que ele foi revogado. Quem conferir a fonte encontra o artigo citado na página indicada, sem nada que avise que ele já não vale. O mesmo aconteceu com o limite de dispensa de licitação para obras (a resposta citou a lei revogada, na página certa dela; a página da lei vigente não chegou às fontes). Nos dois casos o erro começou na busca: na pergunta sobre a tomada de preços, nenhuma página da Lei 14.133 chegou às fontes; na de dispensa, chegaram páginas da Lei 14.133, mas não a do artigo pedido. Nas três perguntas divergentes em que as duas leis chegaram às fontes, o sistema acertou.

Das quatro respostas contadas como erradas, uma estava correta ("até 10 anos", art. 108 da Lei 14.133), mas o gabarito exigia "10 (dez) anos" ou "dez anos"; o número não foi corrigido, e o acerto oficial continua 15 de 19. A quarta, sobre quem conduz a licitação, foi uma recusa: a página-alvo não chegou às fontes, e o sistema respondeu que a informação não constava nos documentos. Com uma repetição e 19 perguntas, o resultado é descritivo.

8.7. Bateria em três domínios

A bateria tem três acervos (segurança do trabalho, produto e consumidor, licitações), com perguntas escritas depois do desenvolvimento. O de licitações repete as duas leis do

anterior e acrescenta a Lei 10.520, de modo que os dois não são independentes. São 71 perguntas: 31 pares de literal e paráfrase (Seção 7) e 9 negativas sem par, cada uma feita três vezes (213 respostas).

Com a pergunta original, a taxa de acerto foi de 68,1%, cerca de 7 em cada 10 respostas; os intervalos de confiança estão no Anexo A7. Na busca, a página certa esteve entre as fontes em 108 de 186 respostas com página-alvo (58,1%, com a métrica corrigida depois da medição; Anexo A8). Das 186 respostas com página-alvo, 118 estavam certas. Em 15 das 118 respostas certas com página-alvo, a página certa não estava entre as fontes: o modelo pode ter respondido pelo que já sabia desses documentos públicos, ou achado a informação em outra página, o que não foi verificado. Respostas certas e com a página certa entre as fontes, que o usuário pode conferir, foram 103 de 186 (55,4%).

Esses números usam o gabarito revisado depois da primeira medição. Quando a pergunta tem resposta, o gabarito revisado deixou de aceitar recusas e respostas que dizem o contrário; quando a pergunta parte de uma premissa falsa, passou a exigir o dado correto. Aplicada às mesmas respostas, a revisão tornou o critério mais rigoroso, e o acerto caiu de 81,7% para 68,1% (Anexo A8). O sistema não mudou. Nas 12 perguntas de existência, a resposta certa é sempre "sim". Responder "Sim." a todas acertaria as 36 respostas desse tipo; o sistema acertou 30. A premissa falsa foi a forma mais difícil: em só 13 de 36 respostas o sistema corrigiu o dado errado.

Redação da pergunta. Por resposta, o acerto foi 88,2% nas literais e 38,7% nas paráfrases. Contando cada versão como certa quando acertou nas três repetições, dos 31 pares, 15 foram discordantes, todos a favor da literal, diferença estatisticamente significativa ($p < 0{,}001$). Com o gabarito original, eram 8 pares a favor da literal e 2 da paráfrase, sem diferença estatisticamente significativa (Anexo A8). É a fraqueza da busca por palavras descrita na Seção 3.1: quem não conhece o termo técnico tem menos chance de achar o trecho.

8.8. Primeira tentativa: enriquecer o índice

A primeira intervenção acrescentou ao índice as palavras do usuário: para cada página, o LLM gerou um resumo e perguntas que ela responde, escritas como um leigo as escreveria. Por desviar do pré-registro, o experimento é exploratório: usou um só acervo, o de segurança do trabalho, com 12 perguntas (5 pares de literal e paráfrase e 2 negativas) e uma repetição, e foi refeito, com adendo ao pré-registro, depois de um erro de implementação. As fontes mudaram entre os braços em 8 das 12 perguntas, mas o acerto quase não mudou. Os braços só com o conteúdo e com as perguntas geradas acertaram 7 de 12. O braço com o resumo, que é o padrão do sistema, acertou 8 de 12, assim como um quarto braço, exploratório, que fica com a maior nota entre o conteúdo e as perguntas geradas.

8.9. Segunda tentativa: expandir a pergunta

A segunda intervenção leva as palavras do documento à pergunta, como em Jagerman et al. [2023]: o modelo recebe só o texto da pergunta e devolve termos técnicos relacionados, que entram na busca; ele não vê o acervo nem o gabarito, e é o mesmo que depois responde. A hipótese pré-registrada era que a expansão melhoraria as paráfrases e deixaria as literais quase iguais. Para a pergunta "Quem paga o capacete e a luva que eu uso no serviço?", que errava nas três repetições, a expansão gerou *equipamento, proteção,*

individual, EPI, fornecedor, empregador, custo, pagamento, e a resposta passou a citar a página certa da NR-6.

Nas mesmas 71 perguntas da bateria, o acerto passou de 68,1% para 88,7% (Tabela 7): o das paráfrases subiu de 38,7% para 83,9% e ficou perto do das literais, que passou de 88,2% para 90,3%. Das 71 perguntas, 15 passaram a acertar nas três repetições e nenhuma deixou de acertar, ganho estatisticamente significativo ($p < 0{,}001$); com o gabarito original, o ganho também foi significativo (Anexo A8). A expansão agiu na busca: a página certa passou a estar entre as fontes em 153 de 186 respostas com página-alvo (82,3%), contra 108 (58,1%) sem ela. Os *tokens* enviados para responder cresceram 13,4%, e cada pergunta nova exige uma chamada a mais ao modelo.

Tabela 7. Expansão da pergunta nas 71 perguntas da bateria. As três primeiras linhas contam respostas (cada pergunta foi feita três vezes); a última conta perguntas. As 9 negativas, sem par, entram só nas linhas de todas as perguntas.

	Sem expansão	Com expansão
Literais, 31 perguntas (% de acerto)	88,2	90,3
Paráfrases, 31 perguntas (% de acerto)	38,7	83,9
Todas, 71 perguntas (% de acerto)	68,1	88,7
Perguntas certas nas três repetições	48 de 71	63 de 71

O controle é a execução anterior da bateria, no mesmo dia, sem sorteio entre as condições, e a versão do modelo no provedor não foi fixada, de modo que pode ter mudado entre as duas. Os três acervos são documentos públicos, que o modelo provavelmente viu no treinamento: "empregador", gerado para a pergunta sobre quem paga o equipamento, é ao mesmo tempo uma ponte de vocabulário e a própria resposta, e o ganho não pode ser estendido a documentos internos de uma organização. Também não houve comparação com expansão sem LLM nem com busca híbrida. Como a primeira ocorrência de cada pergunta depende do modelo, a expansão ficou desligada por padrão; a partir da segunda, o resultado sai de um *cache* e a busca volta a ser determinística. Por isso, as três repetições usaram a mesma expansão e medem a variação da resposta, não a da expansão.

8.10. Edital municipal (exploratório)

O sistema também foi aplicado ao edital de 52 páginas de um concurso público municipal, com 18 perguntas em nove pares de literal e paráfrase, e acertou 15 de 18. As duas versões da pergunta que pede o total de vagas, somando uma tabela de três páginas, erraram: a busca trouxe só parte das páginas, e o modelo somou o que viu na versão literal e disse não ser possível calcular na paráfrase. Isso decorre do número fixo de fontes por resposta (Seção 6.5). A terceira falha foi de busca: a pergunta pelo "Capítulo XVI" não trouxe a página do capítulo, porque o numeral romano tem peso baixo no BM25.

9. Discussão

Resposta à pergunta de pesquisa. A Introdução desdobrou a pergunta em três condições. A Tabela 8 resume o que os dados dizem sobre cada uma.

Tabela 8. Resposta à pergunta de pesquisa, condição por condição.

Condição	Resposta	Por quê
(a) Toda fonte mostrada tem origem lida do banco	Sim	A garantia vem do esquema e dos testes: o documento é ligado por chave estrangeira, conferida por dois testes de integração, e a página mostrada é a gravada na indexação e lida do banco (Seção 6.3). A restrição vale desde a correção descrita nessa seção; antes, só a aplicação garantia o vínculo. Em C1, as 155 unidades têm origem completa no banco; 85 delas são metadados do documento inteiro, sem página. Essa medida confirma que a origem está preenchida, não que a página esteja certa num PDF real. O esquema exige o documento, mas não a página; por isso o objetivo 1 foi atingido só em parte (Seção 10).
(b) A busca por palavras sobre as partes guardadas em tabelas, com a nota calculada na aplicação e sem vetores, acha o trecho certo em perguntas novas	Em parte	Na bateria, a página certa esteve entre as fontes em 58,1% das respostas com página-alvo, e o acerto da resposta foi de 68,1%; no acervo de desenvolvimento, a página certa esteve entre as fontes em 195 de 201. No *corpus* de controle, o BM25 pôs o texto certo em primeiro em 13 de 16 consultas, e a busca densa, em 11 de 16. Com as palavras do documento, o acerto foi de 88,2%; com outras palavras, 38,7%. A expansão por LLM leva a página certa a 82,3% e o acerto a 88,7% (experimental). Não havia acerto mínimo fixado antes da medição; o "Em parte" é um julgamento sobre esses números.
(c) Conferir a fonte confirma que a resposta está certa	Não	A lei revogada é citada com a página certa: quem confere a fonte encontra o texto citado, sem aviso de que ele foi revogado. O sistema também não liga cada frase à fonte que a sustenta, e em 15 das 118 respostas certas da bateria com página-alvo a página certa não estava entre as fontes.

Esses resultados valem na escala medida: os três acervos da bateria têm de três a cinco documentos cada, e as perguntas são em português. Foi possível guardar as unidades e a origem de cada fonte só com o modelo relacional, sem banco de vetores; a nota da busca é calculada na aplicação, sobre essas tabelas. Conferir a fonte, porém, não garante que a resposta esteja certa (Seção 8.6).

O que a garantia de origem cobre. Com o documento inteiro, o modelo citou a página certa tantas vezes quanto com a busca (189 de 201 respostas nos dois casos, aceitando também o número impresso na página). A diferença é que, com a busca, o leitor recebe também uma página lida do banco, que não depende do que o modelo escreveu. Essa garantia vem de guardar cada unidade junto com a sua origem. Se a busca fosse por vetores guardados numa coluna da mesma tabela, a garantia seria a mesma, pois decorre da forma de armazenamento. Como as duas buscas ficaram próximas em C1 e o IVFFlat

na configuração recomendada foi estável até 20.000 vetores sintéticos (o HNSW com parâmetros padrão variou a partir de 5.000), dispensar o banco de vetores não custou acerto detectável em C1 e deixou um único banco a operar. Em C1, os dois tipos de busca erraram em consultas quase sempre diferentes, mas a combinação dos dois acertou tantas quanto o BM25 (13 de 16); pode valer testar a busca híbrida dentro do próprio PostgreSQL num acervo maior.

Comparação com a literatura. O BM25 ficou à frente da busca densa em C1, como no BEIR [Thakur et al. 2021], mas a amostra é pequena demais para confirmar a tendência. A expansão por LLM ajudou, como em Jagerman et al. [2023], e o ganho sobre um buscador simples como o BM25 é coerente com Weller et al. [2024], que observaram que a expansão ajuda sobretudo os buscadores mais fracos. Li et al. [2024] encontraram mais acerto com o documento inteiro; aqui houve empate (71 de 72 nos dois braços), num acervo pequeno e já ajustado, e a vantagem de custo da recuperação se repetiu. No acervo de legislação, os dois erros com a lei revogada começaram na busca: a página da lei vigente com a resposta não chegou às fontes. Por isso esses casos não mostram o conflito entre fontes descrito por Xu et al. [2024], que exigiria as duas páginas certas no *prompt*. A dificuldade com premissas falsas (13 de 36 corrigidas) combina com a tendência do modelo a concordar com quem pergunta [Sharma et al. 2024].

Limitações medidas. A redação da pergunta teve o maior efeito, e a expansão reduziu boa parte dele. Pelo erro de vigência (Seção 8.6), a versão atual não é adequada a acervos que misturam normas vigentes e revogadas. A versão atual também não atende a perguntas cuja resposta soma linhas de muitas páginas, porque o número de fontes é fixo, nem a coleções acima do limite de carga, em que as unidades dos últimos documentos enviados saem da busca sem aviso ao usuário.

9.1. Ameaças à validade

Amostras pequenas. C1 tem 16 consultas com resposta, pouco para qualquer comparação de busca ser significativa. A bateria tem 71 perguntas, e as duas versões de um par não são independentes. Os acervos de legislação e do edital têm uma repetição. O acervo de desenvolvimento (ensino) chegou a 98,6%, resultado que não se repetiu em nenhum acervo de teste e que indica que o ajuste nele inflou o número. As medições corrigidas depois de feitas e o efeito de cada correção estão no Anexo A8.

Um único anotador. O sistema foi ajustado no acervo de ensino. Todos os gabaritos foram escritos por uma só pessoa, sem um segundo anotador nem medida de concordância entre anotadores, e as paráfrases foram escritas por quem já tinha lido o documento. O critério por expressão regular pode aceitar resposta incompleta e recusar resposta certa redigida de outro modo. Casos dos dois tipos foram encontrados e corrigidos na bateria e no acervo de ensino (Anexo A8); no acervo de legislação, uma resposta certa marcada como errada ficou sem correção (Seção 8.6).

O que se mediu. A página nas fontes e a página no texto não medem se o trecho sustenta a frase da resposta. A origem no índice foi medida no *corpus* de controle, que não tem paginação, e a região do parágrafo não foi avaliada em separado.

Conteúdo dos documentos. O texto dos PDFs vai ao modelo junto com a pergunta. Um documento com instruções escritas para o modelo poderia alterar a resposta (injeção de *prompt*), e isso não foi testado.

Generalização. Todos os documentos estão em português, e o maior acervo tem 10 PDFs. A busca densa só foi comparada no *corpus* de controle; nos acervos reais não houve um braço com vetores. Um único modelo redigiu as respostas; um teste exploratório com outro modelo não pôde ser comparado, porque a maioria das respostas marcadas como erradas usa caracteres de espaço e de hífen que o corretor automático do gabarito não reconhece (tabelas de apoio no repositório, Apêndice A). Os acervos de teste são públicos; o comportamento com documentos internos não foi medido. A utilidade para usuários reais não foi avaliada.

10. Conclusão

Este trabalho construiu o ThingIndex, um sistema que guarda a estrutura dos PDFs em tabelas relacionais e responde perguntas mostrando o documento e a página de cada fonte, e o avaliou em sete acervos. Foi possível guardar e mostrar a origem de cada fonte usando só o modelo relacional, sem banco de vetores; a busca por palavras atendeu em parte; e conferir a fonte não bastou para confirmar a resposta (Tabela 8). Os principais resultados foram:

- Cada unidade é guardada numa tabela junto com a sua origem, e a página mostrada ao usuário é lida dali, sem depender do texto do modelo; o vínculo com o documento é garantido por chave estrangeira, e as 155 unidades de C1 têm a origem gravada no banco. O mesmo esquema comportaria vetores numa coluna dessas tabelas.
- Nos acervos com perguntas escritas depois do desenvolvimento, a taxa de acerto foi de 68,1% com a pergunta original. As paráfrases acertaram 38,7% e as literais, 88,2%, e a expansão da pergunta elevou o acerto a 88,7% (experimental).
- Na mediana das perguntas com resposta, buscar trechos usou cerca de 9 vezes menos *tokens* que entregar o documento inteiro ao modelo para responder, sem contar os resumos gerados uma vez na indexação, e sem perda de acerto no acervo de desenvolvimento (71 de 72 perguntas contra 71 de 72, com configurações diferentes entre os dois braços; Seção 8.5).
- Busca por palavras e busca por significado ficaram próximas no *corpus* de controle (13 de 16 consultas contra 11 de 16); dispensar o banco de vetores foi uma escolha de simplicidade de operação, sem ganho de qualidade demonstrado.
- O envio de documentos usa *outbox* transacional e fila com novas tentativas; o teste de caos derrubou a fila, o banco e o armazenamento, e o sistema se recuperou sem reinício manual; 20 envios simultâneos foram todos indexados. A chave estrangeira entre as unidades e o documento é conferida por dois testes de integração.
- O erro mais grave foi a citação da Lei 8.666, já revogada, em perguntas sobre licitações (Seção 8.6).

O objetivo geral foi atingido: o sistema foi projetado, implementado e avaliado, e sua recuperação foi comparada com a busca densa, só no *corpus* de controle, e com o documento inteiro, só no acervo de desenvolvimento. Quanto aos objetivos específicos, o 1 foi atingido em parte: o esquema exige o vínculo de cada unidade com o documento, mas a página é gravada na indexação e, fora da tabela de páginas, pode ficar vazia. A condição (a) é atendida mesmo assim, porque a página mostrada é a gravada e lida do banco; o que falta é o esquema exigir essa página.

Os objetivos 2 a 7 foram atingidos, com ressalvas. No 3, um único anotador escreveu os gabaritos, e eles foram corrigidos depois de medir (Anexo A8). No 4, a

comparação com a busca densa ficou no *corpus* de controle, e no 5, a comparação com o documento inteiro ficou no acervo de desenvolvimento. O 6 usou seis acervos de PDFs reais, e foi nele que apareceu o erro que responde à condição (c). No 7, as duas intervenções foram testadas. A expansão reduziu o efeito da redação da pergunta, e o enriquecimento do índice, ligado por padrão, quase não mudou o acerto no teste exploratório (7 a 8 de 12 perguntas, conforme o braço; Seção 8.8). A vigência, causa do erro mais grave, foi diagnosticada, mas nenhuma intervenção foi testada para ela.

Para a vigência, o modelo relacional oferece um caminho: registrar o período de validade de cada artigo do documento, porque a revogação pode ser parcial, e comparar esses períodos em SQL, que tem recursos próprios para dados temporais [Kulkarni e Michels 2012]. Os trabalhos futuros, em ordem de prioridade, são: tratar a vigência por unidade do documento; avaliar o sistema com usuários reais, medindo o tempo até a resposta e se as fontes são de fato conferidas; medir se o trecho exibido sustenta a frase da resposta; ampliar os gabaritos, com um segundo anotador e documentos que o modelo não conhece; e comparar a expansão e a busca híbrida no próprio PostgreSQL.

O código, os gabaritos, os resultados brutos, os *scripts* que geram e conferem os números deste artigo e os anexos estão no repositório do projeto, `https://github.com/EduardoMG12/ThingIndex`.

Declaração sobre o uso de inteligência artificial generativa

Durante o desenvolvimento do sistema e a preparação deste trabalho, o autor utilizou as ferramentas de inteligência artificial generativa Claude (Anthropic) e ChatGPT (OpenAI) para auxiliar na programação, inclusive dos *scripts* de análise e de conferência dos resultados, na revisão da escrita acadêmica, na sugestão de estruturação das seções, na tradução do resumo para o inglês e na melhoria geral da redação, incluindo a adequação ao limite de páginas. A pergunta de pesquisa, o desenho dos experimentos, as hipóteses, os gabaritos e as decisões técnicas são do autor. Após o uso dessas ferramentas, o autor revisou e editou todo o conteúdo gerado conforme necessário, assumindo plena responsabilidade pelo conteúdo publicado. Os números reportados vêm dos arquivos de resultado versionados e são conferidos por *scripts*.

Referências

Buneman, P., Khanna, S., e Tan, W.-C. (2001). Why and where: A characterization of data provenance. In Van den Bussche, J. e Vianu, V., editors, *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330, Berlin, Heidelberg. Springer.

Chase, H. (2022). LangChain. Repositório de software. Disponível em: `https://github.com/langchain-ai/langchain`. Acesso em: 27 set. 2026.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387.

Cormack, G. V., Clarke, C. L. A., e Buettcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. ACM.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., e Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130.

Es, S., James, J., Espinosa Anke, L., e Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158. Association for Computational Linguistics.

Furnas, G. W., Landauer, T. K., Gomez, L. M., e Dumais, S. T. (1987). The vocabulary problem in human-system communication. *Communications of the ACM*, 30(11):964–971.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., e Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997.

Hevner, A. R., March, S. T., Park, J., e Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1):75–106.

Jagerman, R., Zhuang, H., Qin, Z., Wang, X., e Bendersky, M. (2023). Query expansion by prompting large language models. arXiv preprint arXiv:2305.03653.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., e Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.

Kane, A. (2026). pgvector: Open-source vector similarity search for Postgres. Repositório de software, versão 0.8.6. Disponível em: `https://github.com/pgvector/pgvector`. Acesso em: 27 set. 2026.

Kulkarni, K. e Michels, J.-E. (2012). Temporal features in SQL:2011. *ACM SIGMOD Record*, 41(3):34–43.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., e Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M. F., e Lin, H., editors, *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.

Li, Z., Li, C., Zhang, M., Mei, Q., e Bendersky, M. (2024). Retrieval augmented generation or long-context LLMs? A comprehensive study and hybrid approach. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 881–893.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., e Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Manning, C. D., Raghavan, P., e Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press, New York.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Peffers, K., Tuunanen, T., Rothenberger, M. A., e Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77.

Rashkin, H., Nikolaev, V., Lamm, M., Aroyo, L., Collins, M., Das, D., Petrov, S., Tomar, G. S., Turc, I., e Reitter, D. (2023). Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840.

Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications, Shelter Island.

Robertson, S. e Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askell, A., Bowman, S. R., Cheng, N., Durmus, E., Hatfield-Dodds, Z., Johnston, S. R., Kravec, S., Maxwell, T., McCandlish, S., Ndousse, K., Rausch, O., Schiefer, N., Yan, D., Zhang, M., e Perez, E. (2024). Towards understanding sycophancy in language models. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.

Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., e Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In Vanschoren, J. e Yeung, S., editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1.

VectifyAI (2025). PageIndex: Document index for vectorless, reasoning-based RAG. Repositório de código aberto (licença MIT). Disponível em: `https://github.com/VectifyAI/PageIndex`. Acesso em: 5 set. 2026.

Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., e Wei, F. (2024). Multilingual E5 text embeddings: A technical report. arXiv preprint arXiv:2402.05672.

Weller, O., Lo, K., Wadden, D., Lawrie, D., Van Durme, B., Cohan, A., e Soldaini, L. (2024). When do generative query and document expansions fail? A comprehensive study across methods, retrievers, and datasets. In *Findings of the Association for Computational Linguistics: EACL 2024*, pages 1987–2003.

Xu, R., Qi, Z., Guo, Z., Wang, C., Wang, H., Zhang, Y., e Xu, W. (2024). Knowledge conflicts for LLMs: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8541–8565. Association for Computational Linguistics.

Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., e Zhou, M. (2020). LayoutLM: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1192–1200. ACM.

Apêndice A: Anexos no repositório

Conforme orientação do colegiado, os anexos estão em PDF na pasta `anexos/` do repositório do projeto, `https://github.com/EduardoMG12/ThingIndex`, com um índice em `anexos/README.md`:

- A1 – requisitos funcionais e não funcionais;

- A2 – diagrama de casos de uso;
- A3 – diagrama de sequência do envio de um documento e de uma pergunta;
- A4 – recorte do esquema com o caminho da proveniência;
- A5 – fluxo da recuperação;
- A6 – telas do protótipo;
- A7 – resultados do teste de busca (C1) explicados, com a tabela completa de medidas e os índices aproximados do pgvector, e como os números do artigo foram testados (testes, intervalos de confiança e correção para muitas comparações); a lista das 90 comparações pareadas está nos arquivos `retrieval-significance` de `docs/article/tables` (75 entre textos e 15 entre seções);
- A8 – registro das alterações feitas depois de medir;
- A9 – esquema completo do banco, com as 25 tabelas;
- A10 – uma pasta por acervo, com os PDFs indexados, exceto o do edital municipal, que não está versionado, e um arquivo com as perguntas daquele acervo, a resposta esperada e a página-alvo;
- tabelas de apoio dos resultados: acerto e página no texto por tipo de pergunta no acervo de ensino, com médias e faixa de *tokens*; legislação por tipo de pergunta; bateria por acervo e por forma da pergunta; C1 por tipo de consulta; detalhes da expansão. Estão no arquivo `apoio-resultados.tex`, na pasta `docs/article/tables`.

Os dados brutos, os pré-registros e os *scripts* de conferência estão nas pastas indicadas no índice.

Apêndice B: Pontuação da recuperação

A nota de cada unidade parte do BM25 ($k_1 = 1{,}2$; $b = 0{,}75$), calculado separadamente para cada tipo de unidade da coleção, multiplicado por 100 e arredondado. A nota é então multiplicada por um peso fixo por tipo: 3 para seções, 2 para páginas e trechos e 5 para linhas de tabela. As seções recebem ainda um acréscimo em degraus conforme a confiança da extração e um acréscimo fixo quando não são de primeiro nível. Toda fonte, seja seção, página ou linha de tabela, precisa conter pelo menos dois termos significativos da pergunta (ou o único, se a pergunta tiver um só); nas linhas de tabela, isso impede que um número solto traga uma tabela inteira. Linhas de tabela com menos da metade da nota da melhor são descartadas. Os pesos e limites foram escolhidos durante o desenvolvimento, pelo efeito nas perguntas do acervo de ensino; nenhum deles foi testado isoladamente, retirando-o para medir seu efeito; o código está em `api/internal/usecase/collection_retrieval.go` e, para a seleção das fontes, em `chat_usecase.go`, na mesma pasta.